DefendIt System Guide and Documentation

Thesis Consultation and Milestone Tracker

1 Project overview

DefendIt supports student thesis groups and faculty advisers through a shared consultation and progress workflow. Students organize their group, propose titles, book consultations, and complete milestone tasks. Advisers accept groups, review proposals, record consultation outcomes, and assess completed milestones. This guide explains the user experience, architecture, database, installation process, and essential acceptance tests for the application.

Purpose and problem addressed

Thesis coordination often depends on scattered chat messages, paper notes, and remembered agreements. Groups can lose revision instructions, miss deadlines, or distribute work unevenly. Advisers need a clear record of commitments and a practical view of progress across their assigned groups. DefendIt connects each consultation to notes, assigned action items, and subsequent milestone work so both sides can follow the same record.

Primary feature set

- Google sign-in, persistent sessions, and one-time Student or Adviser role selection.
- Adviser verification through an unused access code and student registration through a unique student number.
- Group creation with a section, thesis topic, and two to five roster members; teammate joining through a six-character code.
- Adviser requests and acceptance, with a maximum caseload of four groups per adviser.
- Title proposals with descriptions, purposes, review decisions, and adviser comments.
- Published consultation slots, protected booking, cancellation, notes, and assigned action items.
- Six thesis milestones with tasks, assignees, due dates, progress, and adviser review.
- A Today dashboard and local SQLite storage with synchronization to Cloud Firestore.

Scope and topic exclusivity

The app topic is thesis advising through consultation scheduling, title review, and milestone tracking. Its defining workflow links consultation outcomes to accountable student work. Topic approval remains subject to instructor confirmation; the documentation does not establish that another group has not registered a similar topic. Confirm the topic, core concept, and primary feature set before submission.

Delivery boundary

Flutter provides the cross-platform application architecture. The documented executable is an Android release APK; Android is the configured and tested target described by the project source. An iOS release requires platform configuration and a macOS build environment. The remaining pages cover personas, interface specifications, wireframes, architecture, authentication, schemas, setup, and practical test cases.

2 Target users and main workflows

The primary roles are Student and Adviser. A student group leader creates the group and initially acts for its roster. A teammate may be listed without having an account; an independently signed-in teammate joins using the group code. These are workflow distinctions within the Student role, not additional authentication roles.

User	Main goal	Permissions and responsibilities
Student leader	Coordinate the thesis group	Create one group, enter its roster, propose titles, book consultations, manage tasks, and submit completed milestones.
Student teammate	Complete assigned work	Join one group, view its shared records, contribute title ideas and milestone tasks, and tick action items.
Adviser	Guide and assess groups	Accept requests, review titles, publish slots, record notes and action items, set milestone due dates, and approve or request revisions.

Student journey

Sign in with Google, select Student, and register a unique student number. Enter a teammate code during onboarding to join an existing group, or proceed to Groups to create or join one. The leader provides a group name, section, thesis topic, and two to five unique roster names, with the leader first. Each student belongs to only one group.

After an adviser accepts the group, students propose titles, read review comments, and revise proposals when needed. They choose an available slot belonging to their adviser and confirm the booking. After consultation, they follow the notes and complete assigned action items. For each milestone, students create and assign tasks, finish them, and submit the milestone when all tasks are done.

Adviser journey

Sign in with Google, select Adviser, and claim a valid unused access code. Open Requests, preview a group and its proposed titles, and accept it while below the four-group limit. Review each title, publish future consultation availability, and record notes during or after booked consultations. Use Progress to inspect group work, set due dates, and approve submissions or request revision with a comment.

Permission boundaries

Students do not approve their own milestones or write adviser review decisions. Advisers do not manage student milestone tasks. Tasks are locked while their milestone is under review or approved. Only the adviser creates or deletes consultation action items; students update completion. Console management of access codes is an administrative maintenance activity, not an in-app user role.

3 UI and UX design specifications

The interface must be custom, cohesive, and intuitive. DefendIt uses a consistent visual identity across login, onboarding, dashboard, lists, forms, and review screens. Flutter widgets provide implementation building blocks, but an unstyled default Material template is not an acceptable finished interface.

Element	Design specification
Palette	Deep indigo #2B2F77 for primary actions, amber #F2A93B for highlights, warm paper white #FAF8F3 for backgrounds, green for success, and red for errors.
App typography	Source Serif 4 headings and Inter body text. The printed guide uses Century Gothic at 11 points.
Components	Rounded cards, labeled status chips, progress bars, bottom sheets, bottom navigation, and a synchronization status chip.
Feedback	Inline validation, readable errors, confirmation dialogs for destructive actions, and visible saved or updated states.

Layout and navigation

After authentication and role registration, the main shell provides Today, Schedule, Groups, and Progress tabs. Keep navigation order, page titles, card styling, and primary action placement consistent. Show role-appropriate actions so students see booking and submission controls while advisers see availability and review controls.

Today emphasizes the next consultation, summary counts, open action items, and Needs attention links. Schedule separates availability from bookings. Groups presents membership, requests, and title proposals. Progress leads from group or milestone summaries into task details and workload. A consultation detail screen opens from a booking or dashboard link.

Forms and keyboard behavior

Use persistent labels and short hints. Keep related fields together and identify validation beside the affected field. Adviser codes use uppercase hyphenated text such as CPE-2026-C3D4; student numbers use their required year-and-number pattern. Section entry should remain editable. Onboarding content should scroll above the keyboard without scaling the entire form down. Preserve readable text sizes and a reachable Continue action.

Screen states

Loading views identify ongoing retrieval. Empty views explain the next step, such as creating a group or publishing availability. Error views offer a readable message and a retry where appropriate. A successful save updates the visible record. Status labels must distinguish pending review, revision, approval, booking, and synchronization rather than depending on color alone. Keep displayed values intact after validation errors so users can correct the form without repeating unrelated input.

4 Wireframes and screen behavior

The following compact wireframes describe information order from top to bottom. Each row specifies a screen rather than a finished screenshot. Apply the visual system from Section 3 when implementing or reviewing the screens. Student and adviser variants share navigation and layout conventions while exposing different permitted actions.

Screen	Content order and principal action
Login	DefendIt identity → short purpose statement → Google sign-in button → inline authentication feedback.
Role selection	Identity → role explanation → Adviser and Student cards → selected role fields → Continue.
Today	Greeting and sync chip → next consultation → counts → Needs attention → open action items → upcoming consultations.
Student Groups	Group summary or empty state → create or join action → roster and adviser information → title ideas.
Adviser Groups	My groups count out of four → group cards → Requests tab → preview and acceptance action.
Request preview	First proposed title or group name → section and topic → roster → up to three proposed titles → Accept.
Title ideas	Proposal cards and status chips → title details → student add or edit sheet, or adviser review sheet.
Schedule	Available slots and bookings → date and time cards → booking confirmation or adviser slot controls.
Consultation	Meeting details → notes → action items with assignees and dates → permitted save or completion action.
Milestone list	Six milestones → status and task progress → due dates → milestone detail navigation.
Milestone detail	Milestone status → tasks → assignees and deadlines → workload → submission or review actions.

Interaction examples

A student opening an unassigned group sees that an adviser has not yet accepted it and cannot book an unrelated adviser's slot. An adviser at the four-group limit sees the current caseload without an active acceptance path. A title requiring revision shows the review comment beside its status so the next action is clear.

For milestone submission, show task completion before the submission action. When the milestone enters review, disable task editing consistently across list and detail controls. On request revision, show the adviser comment and restore the appropriate student work path. These transitions should be visible immediately after the permitted update.

Use the installed application's screenshots as final visual evidence. Wireframes define layout and behavior; they do not substitute for checking actual keyboard handling, text wrapping, empty states, and navigation on the target Android device.

5 System architecture and technology stack

DefendIt separates presentation, repositories, and data services. Screens display application state and call repositories. Repositories apply feature rules and coordinate local or remote persistence. Provider supplies state, and go_router redirects between login, role onboarding, and the main application according to session and profile state.

High level data flow

For offline-capable records, the flow is: user action → repository validation → SQLite record and outbox entry → synchronization service → Cloud Firestore. Firestore listeners deliver remote changes back into SQLite, which notifies the interface. Failed outgoing operations remain queued for retry. Local storage and the outbox update together in a local transaction.

Group creation, joining, adviser acceptance, slot publication, and booking require online coordination. Their repositories use Firestore and transactions because several users may compete for the same shared state. These operations are not promised as offline actions. A consultation also requires an initial online opening before later offline use.

Package or service	Responsibility
Flutter and Dart	Cross-platform screens, interaction, and application code.
firebase_core and firebase_auth	Firebase initialization, Google credential authentication, and persisted sessions.
google_sign_in and cloud_firestore	Account selection, shared records, transactions, and live updates.
sqflite and path	Per-user local database storage and file paths.
provider and go_router	State propagation and session-aware navigation.
connectivity_plus and uuid	Connectivity-triggered synchronization and offline record identifiers.
google_fonts and intl	App typography and readable date formatting.

Source organization

lib/core contains the theme, router, shared widgets, constants, and utilities. lib/features groups screens by authentication, home, groups, titles, slots, consultations, and milestones. lib/data/models defines record structures; repositories hold feature operations; local holds SQLite tables and the outbox; sync coordinates synchronization; remote provides network checks.

Synchronization limits

The source describes merged outgoing changes, timestamp-based last-write handling, and protection for records with unsynchronized local changes. A device with an incorrect clock can affect conflict ordering. A permanently rejected operation may remain in the outbox. The sync chip reports Offline with pending changes, Syncing, or Synced, and supports a manual retry. A successful local save therefore does not by itself prove that another device has received the change.

6 Authentication and session management

DefendIt uses Google sign-in through Firebase Authentication. Phone authentication is an allowed alternative in the assignment, but it is not the selected project method. The signed-in Firebase identity provides the user identifier used for profile ownership and group relationships. A Google account alone does not grant Adviser permissions.

Firebase authentication setup

Create a Firebase project and register the Android application using its actual package identifier. Enable Google in Authentication and select a support email. Register the signing certificate SHA-1 for the build being installed. Generate the Flutter Firebase configuration and place the Android configuration file in android/app/google-services.json. Supply the project's Google Web client ID in its configured application constant.

For development, obtain signing details from the Android Gradle signing report. For release, register the certificate that actually signs the APK rather than assuming it matches a development certificate. The supplied project documentation describes a release configuration using the debug key; verify this setting before distributing a build. Authentication configuration must match both the application identifier and signing certificate.

First registration

After the first successful sign-in, the user selects Student or Adviser once. A student supplies a unique student number, stored in users and indexed by studentNumbers. This is self-declared identity information; the project does not verify it against an official class list.

An adviser enters a one-time access code. The complete hyphenated value identifies a document in adviserCodes. Registration claims the code and creates the matching adviser profile in a transaction. Another account cannot reuse a claimed code. Codes are provisioned privately through the Firebase Console; the source specifies a maximum of fifteen codes. Their existence and availability must be checked during deployment preparation.

Sessions and access control

An authentication-state listener restores the session and directs the user to login, onboarding, or the main shell. Sign-out ends the Firebase and Google sessions. Cancelled account selection and failed authentication must leave a usable login screen with readable feedback.

Firestore rules enforce profile ownership, immutable identity fields, role-specific writes, and group access. The source also identifies limits: the four-group cap, one-group membership constraint, and all-tasks-complete submission gate depend on application logic and transactions rather than complete server-side enforcement. Do not describe these checks as guarantees against a modified client. Access-code maintenance requires a controlled administrative process because leaked unused codes can be claimed.

7 Cloud Firestore schema

Cloud Firestore is the shared source of truth. Document identifiers connect users, groups, schedules, and consultation records. The following field map preserves the supplied project's collection names. Listed fields describe storage responsibilities; optional assignment, review, and booking values may be absent until the relevant workflow occurs.

Collection or path	Fields and mapping
users/{uid}	uid, name, email, photoUrl, role, studentNumber, adviserCode, adviserGroupCount, createdAt. The role is student or adviser.
studentNumbers/{number}	uid, createdAt. The formatted student number is the unique document identifier associated with one account.
adviserCodes/{code}	label, claimedByUid, claimedAt. The complete formatted code is the document identifier.
groups/{leaderUid}	name, section, thesisTitle, code, memberUids, memberNames, roster, leaderUid, adviserUid, adviserName, createdAt.
groups/{id}/titles/{titleId}	groupId, title, description, purpose, status, createdByUid, createdByName, comment, reviewedByName, reviewedAt, createdAt, updatedAt.
groups/{id}/milestones/{groupId_slug}	groupId, slug, title, position, status, dueAt, comment, updatedByName, updatedAt.
groups/{id}/tasks/{taskId}	groupId, milestoneId, title, assignee, dueAt, done, doneAt, doneByName, createdByName, createdAt, updatedAt.
slots/{slotId}	adviserUid, adviserName, startsAt, durationMins, status, bookedGroupId, bookedGroupName, bookedByUid, bookedByName, createdAt.
consultations/{slotId}	groupId, groupName, adviserUid, adviserName, startsAt, durationMins, roster, notes, createdAt, updatedAt.
actionItems/{itemId}	consultationId, groupId, groupName, adviserUid, title, assignee, dueAt, done, doneAt, createdAt, updatedAt.

Relationships and value conventions

A group identifier equals its leader's user identifier. memberUids links signed-in teammates, while the roster also represents named members without accounts. A group has at most one adviser, multiple title ideas, six milestones, and tasks linked through milestoneId. A consultation uses its slot identifier and connects to action items through consultationId.

Identifiers and descriptive values are strings; membership and roster values are lists; completion flags are booleans; counts, positions, and durations are numeric. The source records application times as Unix epoch milliseconds and includes creation and claim metadata. Preserve the implementation's existing serialization when mapping these values; do not silently convert metadata or identifiers to a different representation.

8 Local storage and CRUD behavior

SQLite stores offline-capable records in a separate database for each signed-in account. The documented filename is thesistrack_[uid].db, schema version 4. Consultations, action_items, title_ideas, milestones, and milestone_tasks mirror their corresponding cloud records, adding a local id where the remote identifier is held in the document path. Roster values use JSON text, and local completion values use 0 or 1.

The outbox contains seq, collection, docId, op, data, and createdAt. seq orders pending operations, collection records the remote path, op is upsert or delete, and data contains the JSON payload. Outgoing changes are replayed oldest first; remote listeners update local tables for the current user's authorized groups and advising records.

Entity	Create	Read	Update	Delete
Group	Student leader	Signed-in users	Join or adviser acceptance	Not supported
Title idea	Group member	Authorized members and adviser; open-group adviser preview	Student revision or adviser review	Member if not approved
Milestone	Group initialization	Members and adviser	Student progress; adviser review and dates	Not supported
Task	Group member	Members and adviser	Member edits and completion when unlocked	Group member when unlocked
Slot	Adviser	Signed-in users	Authorized booking or cancellation	Adviser if open
Consultation	Adviser	Assigned adviser and members	Adviser notes	Not supported
Action item	Adviser	Assigned adviser and members	Adviser edits; student completion	Adviser

Status field mapping

Title states are pending, approved, needsPolish, and rejected. Milestone states are notStarted, inProgress, forReview, needsRevision, and approved. Slot states are open and booked. Display readable labels while preserving these stored values. Editing a title needing polish resubmits it as pending. Milestone review or approval locks tasks; requesting revision requires a comment.

Integrity and offline boundaries

Transactions protect competing bookings and adviser acceptance. Complete CRUD is demonstrated through supported entities such as titles, tasks, slots, and action items; it does not imply that every entity supports deletion. Offline work covers previously synchronized records, including notes, title ideas, tasks, and milestone updates. Initial sign-in, shared coordination, and request previews require connectivity. Validate remote persistence after synchronization rather than treating a local display change as cloud evidence.

9 Development setup and installation

Use the project repository and its declared dependency constraints. Prepare Flutter, Android Studio with Android SDK tools, and an Android phone or Google Play-enabled emulator. The supplied project documentation targets Android 6.0 or later. Confirm compatibility against the selected build configuration and device. Use one adviser and two student test accounts to exercise shared workflows.

Environment and Firebase configuration

1. Install Flutter and add its bin directory to PATH. Install the Flutter and Dart IDE plugins, then run flutter doctor and resolve the Android toolchain findings.

2. Open the project root and run flutter pub get. Keep the resolved dependencies consistent with the delivered project rather than upgrading them during submission preparation.

3. Install Node.js and the Firebase CLI where needed, authenticate with firebase login, and activate FlutterFire CLI with dart pub global activate flutterfire_cli.

4. Create or select the Firebase project, register the Android package, enable Google authentication, and add the build's SHA-1 fingerprint. From android, run .\gradlew signingReport on Windows to inspect signing configuration.

5. Run flutterfire configure for the intended Firebase project. Confirm lib/firebase_options.dart, android/app/google-services.json, and the configured Google Web client ID correspond to that project.

6. Create Cloud Firestore, publish the supplied firestore.rules, and provision adviserCodes documents privately. Ensure test users and codes belong to this environment.

Run and build

Connect a device with USB debugging or start the emulator, then run flutter run. Check login, role onboarding, and data retrieval before preparing a standalone package. For a clean release build, run the commands below from the project root in order.

flutter clean

flutter pub get

flutter build apk --release

The APK is generated at build/app/outputs/flutter-apk/app-release.apk. Confirm the main Android manifest declares Internet access. Copy the APK to the device, allow installation from the chosen source, install, and launch without the development machine attached.

Operational checks

If sign-in fails, verify package identity, signing fingerprint, Firebase configuration, and Web client ID. For permission errors, check the deployed rules and user profile. Missing data on a new device may require its first online synchronization. If milestone initialization is missing, use the documented Set up milestones action while online. Do not remove identity records or reset claimed codes casually to bypass registration failures.

10 Essential UI and authentication tests

Run these cases on the installed Android application using a configured test Firebase environment. Prepare an unused adviser code, a claimed code, two distinct student numbers, and the three test accounts described in Section 9. Use fresh accounts for first-registration tests. These are acceptance cases, not claims that execution has already passed.

Test ID	Feature	Input or action	Expected result
UI01	Sign-in view	Launch while signed out.	Custom login appears with Google sign-in and readable text.
AU01	Google login	Select a valid Google account.	Firebase session starts; a new user reaches role selection.
AU02	Cancellation	Cancel account selection.	App remains usable at login without a partial registration.
AU03	Adviser code	Enter a valid unused hyphenated code.	Adviser profile and matching code claim are saved together.
AU04	Claimed code	Register another account with a claimed code.	Registration is rejected with a readable message.
AU05	Student identity	Register a valid unused student number.	Student profile and number ownership record are created.
AU06	Duplicate number	Use a number already owned by another account.	Registration is rejected; ownership does not change.
AU07	Session	Close and reopen after registration.	Existing session resumes at the appropriate app screen.
AU08	Sign-out	Sign out and reopen the app.	Protected screens require authentication again.
UI02	Input formatting	Type a lowercase adviser code continuously.	Uppercase hyphenated code appears and remains the submitted value.
UI03	Keyboard	Focus onboarding fields and scroll.	Fields remain readable and reachable without whole-form squishing.
UI04	Section editing	Type BSCPE 4, then delete the dash and digit.	Hyphen is inserted on typing; spaces and backspace remain editable.
UI05	Navigation	Visit Today, Schedule, Groups, and Progress.	Consistent navigation and role-appropriate actions appear.

Recording results

For each execution, record the test ID, build identifier, date, tester, actual result, and pass or fail outcome in the test log. Attach a screenshot or short reproduction note when the observed result differs. Keep account identifiers and access codes out of shared screenshots. Retest the affected case after a fix and include the result with the submission evidence. No table entry should be marked passed merely because the interface or source code appears to support it.

11 Functional and Firebase CRUD tests

Prepare a student group, an assigned adviser, an open future slot, and a second client for checking live updates. Use authorized test accounts and disposable records. Inspect the matching Firestore document when validating persistence and permissions.

Test ID	Feature	Input or action	Expected result
DB01	Create group	Submit valid group and roster.	Group is saved; leader belongs to the new group.
DB02	Join group	Second student enters teammate code.	Membership updates and the same group opens.
FN01	Adviser acceptance	Preview and accept an open request.	Adviser assignment appears; count respects the four-group cap.
DB03	Title CRUD	Add, read, edit, then delete an unapproved title.	Matching cloud record follows each change after sync.
FN02	Title review	Request polish with a comment; student revises.	Feedback is visible and the revised title becomes pending.
DB04	Task CRUD	Add, read, edit, complete, and delete a task.	Task and progress updates persist and appear to the other client.
FN03	Milestone review	Try incomplete submission; finish tasks and submit.	Incomplete work is blocked; complete work enters locked review.
DB05	Slot CRUD	Publish a future slot, read it, book or cancel, then delete when open.	Each permitted state persists; booked slots cannot be deleted.
FN04	Booking conflict	Two eligible groups attempt the same open slot.	Only one booking succeeds.
DB06	Consultation	Adviser saves notes; student opens the record.	Authorized student sees the saved consultation notes.
DB07	Action item CRUD	Adviser adds and edits item; student ticks it; adviser deletes.	Each permitted change persists and updates the shared view.
DB08	Access control	Unrelated user attempts a protected record update.	Firestore rejects the unauthorized write.
SY01	Offline sync	Edit a synchronized task offline, then reconnect.	Local change stays visible and reaches Firestore after sync.
SY02	Live update	Save an online title edit on another client.	Authorized client receives and displays the updated record.

Submission evidence and limitations

Submit the executable Flutter and Firebase application, this system guide, and the completed test results. Include actual screenshots and the deployed rules as supporting evidence. Record Android validation separately from any future iOS work. The documented limits include online-only coordination, device-clock conflict handling, self-declared student numbers, no group deletion, and no attachments or push notifications. Instructor topic approval remains an external confirmation rather than a test result.